

Vincenzo Rubulotta
Matricola 1000034149

Relazione Homework 3 Prova in Itinere

DSBD

1. Introduzione

Il documento descrive l'evoluzione dell'architettura sviluppata negli step precedenti, focalizzandosi sulla migrazione dell'orchestrazione da Docker Compose a un ambiente Kubernetes gestito con Kind e sull'implementazione di un monitoraggio White-box tramite Prometheus. Questa transizione mira a simulare un deployment Cloud Native reale, assicurando scalabilità, resilienza e osservabilità, con un'attenzione particolare all'automazione dei processi di build e deploy tramite script dedicati.

2. Schema Architetture

L'architettura preserva l'impostazione a microservizi già adottata, riorganizzando però i componenti all'interno di Pod Kubernetes gestiti tramite risorse Deployment e Service. Il traffico utente in ingresso viene gestito in HTTPS dall'API Gateway Nginx, che agisce come unico punto di accesso e smista le richieste ai servizi core user-manager e data-collector. La gestione dei dati si appoggia a PostgreSQL per la persistenza e all'ecosistema Kafka/Zookeeper per il message brokering, mentre l'osservabilità dell'intero stack è assicurata da un server Prometheus dedicato alla raccolta delle metriche.

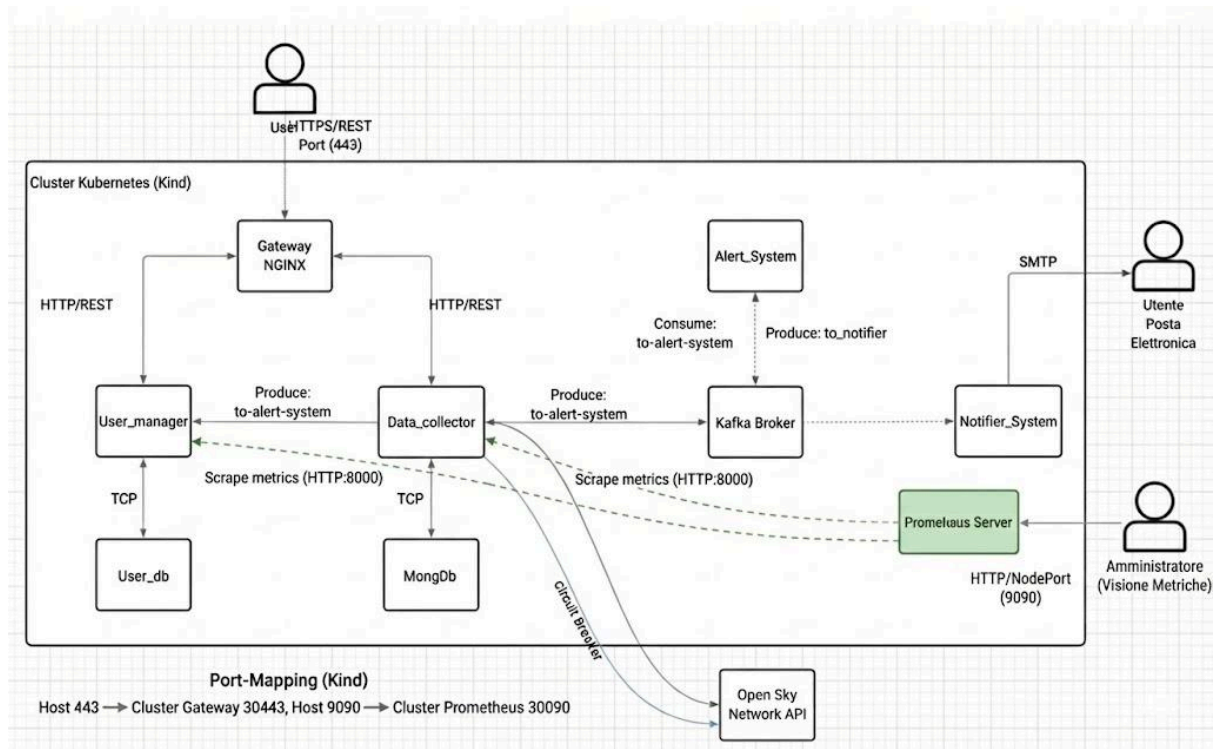


Figura 1 - Diagramma architetturale dei componenti e flussi di comunicazione

3. Scelte Progettuali e Motivazioni

In questa sezione vengono analizzate le decisioni architetturali critiche adottate per l'evoluzione del sistema, motivando le scelte implementative.

3.1 Orchestrazione con Kind e Port-Mapping

La scelta di utilizzare Kind come ambiente di esecuzione ha richiesto un'attenzione particolare alla gestione del networking. Per superare i limiti e l'instabilità del port-forwarding manuale, è stata implementata una configurazione avanzata tramite il parametro `extraPortMappings` nel file `kind-config.yaml`. Questo approccio consente di mappare direttamente sull'host le porte 443 e 9090, rendendo immediatamente accessibili il Gateway e la dashboard di Prometheus; il risultato è un sistema ready-to-use subito dopo il deploy, che elimina completamente la necessità di configurare tunnel aggiuntivi post-avvio.

3.2 Monitoraggio Prometheus

La strategia di monitoraggio abbraccia il pattern White-box: anziché mascherare il server Prometheus dietro l'API Gateway, si è deciso di esporlo direttamente su una porta dedicata tramite la NodePort 30090, mappata internamente sulla 9090. Questa scelta architetturale garantisce una netta separazione logica tra il traffico utente, destinato alla business logic, e quello amministrativo riservato al control plane, migliorando la pulizia e la gestione delle rotte.

3.3 Gestione della Sicurezza

La protezione delle credenziali critiche, come le password del database e le chiavi API, è affidata agli oggetti Secret di Kubernetes. In linea con le best practices DevSecOps, il file di configurazione `00-secret.yaml` è stato escluso dal versionamento tramite `.gitignore`, eliminando così il rischio di esposizione accidentale o leakage di dati sensibili nel repository.

Il file è necessario per il corretto funzionamento del sistema, verrà quindi spiegato come costruirlo nel file `.md` contenuto all'interno del versionamento.

4. Dettagli delle metriche Prometheus

L'strumentazione dei microservizi è stata realizzata integrando le librerie client di Prometheus, che espongono le metriche su un endpoint HTTP dedicato alla porta 8000. Per garantire una tracciabilità puntuale, i dati raccolti sono stati arricchiti con le label `service` e `node`, quest'ultima valorizzata dinamicamente recuperando il nome del nodo Kubernetes direttamente dalle variabili d'ambiente.

4.1 User Manager

Per quanto riguarda il servizio User-Manager, l'osservabilità è garantita da due metriche principali. Il traffico in ingresso viene monitorato tramite il Counter `user_manager_requests_total`, che tiene traccia del volume cumulativo di richieste di registrazione e login. Parallelamente, per analizzare le prestazioni, è stato introdotto il Gauge `user_manager_operation_latency_seconds`: questa metrica rileva in tempo reale la latenza delle operazioni verso il database, permettendo di identificare con precisione eventuali colli di bottiglia nel sistema.

4.2 Data Collector

Il monitoraggio del Data-Collector si focalizza sui volumi di lavoro e sui tempi di risposta: attraverso il Counter `data_collector_requests_total` viene tenuto il conto complessivo delle iterazioni di raccolta dati effettuate. Parallelamente, il Gauge `data_collector_processing_time_seconds` traccia la durata della fase di elaborazione interna, misurando nello specifico il tempo necessario a preparare i dati prima del loro invio al broker Kafka.

5. Analisi dei Manifest Kubernetes

5.1 Prometheus

La configurazione dei job di scraping è definita tramite una ConfigMap, sfruttando appieno il Service Discovery di Kubernetes. Questo meccanismo permette a Prometheus di individuare automaticamente i servizi target, come user-manager e data-collector, rendendo superflua la gestione manuale degli indirizzi IP dinamici tipica degli ambienti containerizzati. L'accesso alla dashboard è infine garantito da un Service di tipo NodePort configurato sulla porta statica 30090, che apre un canale diretto dall'host senza intermediari. La configurazione dei job di scraping è definita tramite una ConfigMap, sfruttando appieno il Service Discovery di Kubernetes. Questo meccanismo permette a Prometheus di individuare automaticamente i servizi target, come user-manager e data-collector, rendendo superflua la gestione manuale degli indirizzi IP dinamici tipica degli ambienti containerizzati. L'accesso alla dashboard è infine garantito da un Service di tipo NodePort configurato sulla porta statica 30090, che apre un canale diretto dall'host senza intermediari.

5.2 API Gateway

Nginx ricopre il ruolo di Ingress Controller personalizzato, appoggiandosi a un Service di tipo NodePort configurato sulle porte 30080 e 30443. Queste porte corrispondono esattamente al mapping definito nella configurazione del cluster Kind, garantendo la corretta esposizione verso l'host. Oltre al routing, il gateway gestisce interamente la terminazione SSL, utilizzando certificati self-signed montati in sicurezza all'interno del Pod tramite l'uso dei Secret.

5.3 Microservizi e Database

La definizione delle risorse e la configurazione delle variabili d'ambiente sono affidate agli oggetti Deployment, che gestiscono il ciclo di vita dei Pod. Sul fronte del networking, per i componenti puramente interni come il Database, Kafka e i microservizi core, si è optato per l'utilizzo di Service di tipo ClusterIP. Questa configurazione isola i servizi dall'accesso pubblico diretto, restringendo la comunicazione esclusivamente al traffico interno al cluster o a quello mediato in modo controllato dall'API Gateway.